
Trabajo Práctico Final

Materia:
Virtualización y Consolidación de Servidores

Docente:
Ing. Luis María Carriles

Alumna:
Albornoz, Silvia Melisa
DNI: 45 275 151
Legajo: 56113

Índice

1. Introducción	2
2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE	2
3. Preparación del Código en Entorno Local	2
3.1. Estructura del Proyecto	3
3.2. Dependencias de Producción	3
4. Transferencia de Archivos al Contenedor	3
5. Despliegue de la Aplicación Flask (LXC 167)	4
5.1. Aislamiento del Entorno Python	4
5.2. Lanzamiento del Servidor en Segundo Plano	4
6. Configuración de PostgreSQL (LXC 168)	4
6.1. Inicialización del Motor de Base de Datos	4
6.2. Aprovisionamiento de Entidades Relacionales y Credenciales	5
7. Validación del Funcionamiento y Vinculación entre Contenedores	5
7.1. Verificación de Conectividad Inter-LXC	5
7.2. Prueba Funcional del Blog	5
7.3. Imagen de Perfil del Blog mediante Hosting Externo	6
7.4. Lógica de Descarga Forzada del Informe de Fin de Curso	6
8. Conclusión	6

1. Introducción

El presente trabajo documenta el desarrollo e implementación de un **Blog Personal** desplegado sobre una infraestructura basada en contenedores Linux. La solución se compone de una aplicación web desarrollada con **Flask/Python** y una base de datos **PostgreSQL**, ejecutadas en entornos independientes para favorecer la modularidad y la administración de los servicios.

El proyecto fue implementado en la infraestructura del **NAP** (*Núcleo de Asistencia de Redes*) de la facultad, aplicando conceptos de virtualización, configuración de servidores y comunicación entre servicios. Durante el proceso se realizaron tareas de instalación, configuración, pruebas y resolución de inconvenientes técnicos, logrando una plataforma funcional, accesible de forma remota y con persistencia de la información.

2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE

Para garantizar el aislamiento de procesos y optimizar el uso de los recursos del hipervisor, se aprovisionaron **dos contenedores Linux (LXC) independientes** sobre la distribución **Debian 12 Bookworm**, cada uno con responsabilidad única (principio de *single responsibility*):

Parámetro	LXC Flask (Aplicación)	LXC PostgreSQL (BD)
Hostname	45275151A	45275151DB
Plantilla de OS	debian-12-standard	debian-12-standard
vCPU	1 núcleo	1 núcleo
Memoria RAM	128 MB	128 MB
Memoria Swap	128 MB	0 MB
Almacenamiento	8 GB (ext4)	8 GB (ext4)
Interfaz de red	eth0 (Bridge: vmbr0)	eth0 (Bridge: vmbr0)
Dirección IP	172.16.90.167/24	172.16.90.168/24
Puerta de enlace	172.16.90.1	172.16.90.1

Tabla 1: Especificaciones de los contenedores LXC aprovisionados en Proxmox VE.

3. Preparación del Código en Entorno Local

La aplicación web fue desarrollada en Python utilizando el *framework* **Flask** con el driver nativo **psycopg2** para la interacción con la base de datos PostgreSQL.

3.1. Estructura del Proyecto

El repositorio local se organizó respetando la jerarquía estándar de Flask para activos estáticos y plantillas Jinja2:

```
1 blog/
2     app.py
3     static/
4         uploads/
5             Informe_TPF.pdf
6     templates/
7         index.html
8         admin.html
```

Imagen 1: Estructura del repositorio de la aplicación.

3.2. Dependencias de Producción

Las librerías requeridas se fijaron con versiones específicas en el archivo `requirements.txt` para garantizar la reproducibilidad del entorno:

```
1 Flask==2.3.2
2 psychopg2-binary==2.9.6
3 Werkzeug==2.3.6
```

Imagen 2: Contenido de `requirements.txt`.

4. Transferencia de Archivos al Contenedor

Dado que el aislamiento perimetral de la red universitaria imposibilitaba el acceso directo entrante vía SSH/SFTP desde una ubicación externa, se utilizó **GitHub** como pasarela de transferencia intermedia bajo la cuenta de usuario AlbMelisa.

Procedimiento en el equipo local:

```
1 cd ~/blog
2 git init
3 git add .
4 git commit -m "Initial commit: blog personal Flask"
5 git remote add origin https://github.com/AlbMelisa/tfi-blog-
  flask.git
6 git push -u origin main
```

Imagen 3: Inicialización del repositorio y carga a GitHub.

Procedimiento en el contenedor LXC 167 (con salida nativa a Internet a través del puerto 443/HTTPS):

```
1 cd /opt
2 git clone https://github.com/AlbMelisa/tfi-blog-flask.git
```

Imagen 4: Clonado del repositorio dentro del contenedor de aplicación.

5. Despliegue de la Aplicación Flask (LXC 167)

5.1. Aislamiento del Entorno Python

Para evitar conflictos con los paquetes del sistema operativo Debian 12, se instaló la utilidad de entornos virtuales y se inicializó un espacio aislado de ejecución:

```
1 apt install python3-venv -y
2 cd /opt/blog
3 python3 -m venv venv
4 source venv/bin/activate
```

Imagen 5: Creación y activación del entorno virtual Python.

Con el entorno activo (indicado por el prefijo (**venv**)), se actualizaron las herramientas de empaquetado y se instalaron las dependencias del *backend*:

```
1 pip install --upgrade pip
2 pip install -r requirements.txt
```

Imagen 6: Instalación de dependencias en el entorno virtual.

5.2. Lanzamiento del Servidor en Segundo Plano

Para asegurar la persistencia del proceso web de Flask y evitar su finalización al cerrar la terminal interactiva del hipervisor Proxmox, se ejecutó la aplicación utilizando **nohup**, que desvincula el proceso del terminal de control:

```
1 nohup python3 app.py > /opt/blog/flask.log 2>&1 &
2 echo "PID: $!"
```

Imagen 7: Ejecución persistente del servidor Flask.

Nota técnica: La redirección `> /opt/blog/flask.log 2>&1` consolida la salida estándar y los errores en un único archivo de log, facilitando el diagnóstico posterior sin necesidad de mantener una sesión interactiva abierta.

6. Configuración de PostgreSQL (LXC 168)

6.1. Inicialización del Motor de Base de Datos

Desde la consola del contenedor dedicado a persistencia (45275151DB), se instalaron los paquetes binarios del motor relacional:

```
1 apt update && apt install postgresql postgresql-contrib -y
2 systemctl enable postgresql
3 systemctl start postgresql
```

Imagen 8: Instalación de PostgreSQL en el contenedor de base de datos.

6.2. Aprovisionamiento de Entidades Relacionales y Credenciales

Se accedió a la consola interactiva `psql` adoptando los privilegios del superusuario del sistema:

```
1 su - postgres
2 psql
```

Imagen 9: Acceso al intérprete de comandos `psql`.

Dentro del prompt `postgres=#`, se configuraron el usuario de aplicación, la base de datos dedicada, la codificación de caracteres y la zona horaria regional:

```
1 CREATE USER postgres WITH PASSWORD '45275151';
2
3 CREATE DATABASE blog_db
4     WITH OWNER          = postgres
5         ENCODING         = 'UTF8'
6         LC_COLLATE       = 'es_AR.UTF-8'
7         LC_CTYPE         = 'es_AR.UTF-8';
8
9 GRANT ALL PRIVILEGES ON DATABASE blog_db TO postgres;
```

Imagen 10: Creación de usuario, base de datos y configuración regional.

7. Validación del Funcionamiento y Vinculación entre Contenedores

Una vez finalizadas las fases de instalación y configuración, se procedió a verificar el correcto funcionamiento de ambos contenedores y la comunicación efectiva entre ellos.

7.1. Verificación de Conectividad Inter-LXC

Desde el contenedor de aplicación (45275151A, IP 172.16.90.167), se comprobó la conectividad de red hacia el contenedor de base de datos (45275151DB, IP 172.16.90.168) mediante `ping`:

```
1 ping -c 3 172.16.90.168
```

Imagen 11: Prueba de conectividad ICMP entre contenedores.

7.2. Prueba Funcional del Blog

Finalmente, se accedió al blog desde el siguiente enlace:

<https://nap.frt.utn.edu.ar/45275151>

Se verificaron las siguientes operaciones de forma satisfactoria: la visualización de la página principal con las publicaciones cargadas, la creación de nuevas entradas desde el panel de administración (/admin) con persistencia en la base de datos PostgreSQL, la correcta resolución de las URLs internas bajo el subdirectorio institucional, y la descarga del informe PDF mediante el botón dedicado.

7.3. Imagen de Perfil del Blog mediante Hosting Externo

Para la imagen personal mostrada en la interfaz del blog, se utilizó el servicio de alojamiento de imágenes **Postimages** (<https://postimages.org>). La imagen fue subida a la plataforma y se obtuvo una URL de enlace directo, la cual fue referenciada en el código HTML de la plantilla mediante una etiqueta :

```
1 
```

Imagen 12: Referencia a la imagen personal alojada en Postimages.

Esta estrategia permitió incorporar contenido multimedia sin consumir el limitado espacio de almacenamiento del contenedor LXC (8 GB) y sin necesidad de configurar un servicio de archivos estáticos adicional.

7.4. Lógica de Descarga Forzada del Informe de Fin de Curso

Para el botón de descarga del documento técnico integrado en la interfaz del blog, se implementó un endpoint exclusivo en el backend que encapsula la descarga directa como adjunto binario, previniendo que los navegadores fueren la apertura en línea del archivo:

```
1 from flask import send_from_directory
2
3 @app.route("/download-pdf")
4 def download_pdf():
5     return send_from_directory(
6         app.config["UPLOAD_FOLDER"],
7         "Informe_TPF.pdf",
8         as_attachment=True,
9         download_name="TFI_Virtualizacion_AlbornozSilviaMelisa
10         .pdf"
```

Imagen 13: Ruta dedicada a la descarga directa del archivo adjunto.

8. Conclusión

La arquitectura propuesta fue implementada de forma exitosa, cumpliendo con la totalidad de los requisitos funcionales y técnicos definidos para el proyecto. Los principales logros obtenidos fueron:

- **Separación de responsabilidades:** la distribución de los servicios en nodos LXC independientes permitió una gestión más eficiente de los recursos y facilitó el diagnóstico de fallos.
- **Operación dentro de restricciones de hardware:** la aplicación y la base de datos operan de forma estable con tan solo 128 MB de RAM cada una, demostrando la eficiencia del stack seleccionado.
- **Conectividad segura inter-contenedor:** la configuración de PostgreSQL restringe el acceso remoto únicamente a la subred interna 172.16.90.0/24, reduciendo la superficie de ataque.

En síntesis, la infraestructura desplegada constituye una solución confiable, organizada y escalable, capaz de satisfacer los requerimientos actuales del proyecto y de servir como base para futuras ampliaciones, tales como la configuración de un servicio *systemd* para la gestión del ciclo de vida del proceso Flask.